

P0553R3: Bit operations

Introduction

This paper proposes to add simple free functions for basic bit operations on all unsigned integer types.

A previous proposal was in "N3864: A constexpr bitwise operations library for C++" by Matthew Fioravante.

The papers P0237Rx propose to introduce `bit_value`, `bit_reference`, and `bit_iterator`, higher-level concepts for manipulating individual bits and sequences of bits. In order to implement efficient standard algorithms on `bit_iterator`, it is expected that implementations use the facilities presented in this paper. By directly exposing the low-level facilities to programmers, they can use them for their own purposes, independent of higher-level abstractions in the standard library.

It is expected that the functions provided with this proposal will be, at some later time, overloaded for `std::datapar`, the nascent SIMD data type (see P0214R2: "Data-Parallel Vector Types & Operations" by Matthias Kretz).

Changes

P0553R0 was favorably reviewed by SG6 (including the function names) and forwarded to LEWG. There are only editorial changes in the P0553R1 revision of the paper.

P0553R1 was favorably reviewed by LEWG with removing the nested inline namespace and using the header `<bit>`, and forwarded to LWG at the Albuquerque, Fall 2017 meeting.

P0553R2 contains small edits in preparation for LWG review.

P0553R3 reflects LWG review (introduce "Constraints" items, harmonize phrasing "0/1 bits in the value of x", add "nodiscard" for `rotl/rotr`). It was sent back to LEWG to clarify the open issues (see below).

Open issues for LEWG

- Tentative YES: Should the rotation count be signed, and should a negative value mean rotation in the opposite direction? (Geoffrey Romer)
- Tentative YES: "count" should return the same type as the rotation count of `rotl/rotr`: `std::rotr(val, std::countl_zero(val))` (Ville Voutilainen)
- Tentative YES: Is it intentional that the "unsigned" requirement excludes `std::byte` or `char`, even on platforms where `char` is unsigned?

Design considerations

These operations can be made available either in the `<bit>` header proposed by P0237R4, or in a separate header. LEWG decided to use `<bit>`.

The bit rotation operations are not limited to rotation counts less than the bit-width of the integer; it seems most (all?) CPU instruction sets support longer rotations by simply discarding upper bits. Different from bit shifts, this behavior is reasonable because rotating a 32-bit quantity by 33 bits is indistinguishable from rotating it by 1 bit.

The counting operations return "int" quantities, consistent with the rule "use an int unless you need something else". This choice does not reflect, in the type, the fact that counts are always non-negative.

This proposal uses readable English names for the operations, except that "left" (towards/affecting the most significant bits) and "right" (towards/affecting the least significant bits) is abbreviate "l" and "r", respectively. The use of "left" and "right" in the operation names is consistent with the "left" and "right" built-in shift operators.

An earlier draft of this paper had names such as "countl1", but lower-case "ell" and the digit "one" are nearly indistinguishable in some fonts, so now a name like "countl_one" is proposed. This is slightly baroque, so alternatively "countl(T, bool)" could be used, with the understanding that the second parameter is often a compile-time constant, allowing for optimization opportunities.

Per prevailing LWG convention, only those functions are marked `noexcept` that have a wide construct, i.e. no restrictions on the values of the function arguments.

All functions are marked `constexpr`, assuming that either compiler intrinsics that work in constant evaluation are available to the implementation, or the optimizer is good enough to use the appropriate hardware instructions for operations described with potentially verbose expressions with built-in operators.

A tangentially related `std::bit_cast` is proposed in P0476R1: "Bit-casting object representations" by JF Bastien.

Hardware

This is an incomplete table showing the extent of hardware support for the proposed operations.

operation	Intel/AMD	ARM	PowerPC
rotr	ROL	-	rldicl
rotr	ROR	ROR, EXTR	-
popcount	POPCNT	-	popcntb
countl_zero	BSR, LZCNT	CLZ	cntlzd
countl_one	-	CLS	-
countr_zero	BSF, TZCNT	-	-
countr_one	-	-	-

Wording

Add the header `<bit>` to the table in 17.5.1.2 [headers].

Append a new subsection to clause 20 [utilities] with the following content:

23.20 Bit operations [bitops]

23.20.1 Header `<bit>` synopsis [bit.syn]

```
namespace std {
// 23.20.2, rotating
template<class T>
    [[nodiscard]] constexpr T rotr(T x, int s) noexcept;
template<class T>
    [[nodiscard]] constexpr T rotr(T x, int s) noexcept;

// 23.20.3, counting
template<class T>
    constexpr int countl_zero(T x) noexcept;
template<class T>
    constexpr int countl_one(T x) noexcept;
template<class T>
    constexpr int countr_zero(T x) noexcept;
template<class T>
    constexpr int countr_one(T x) noexcept;
template<class T>
    constexpr int popcount(T x) noexcept;
}
```

In the following descriptions, let `N` denote `std::numeric_limits<T>::digits`.

23.20.2 Rotating [bitops.rot]

```
template<class T>
    [[nodiscard]] constexpr T rotr(T x, int s) noexcept;
```

Constraints: `T` is an unsigned integer type (3.9.1 [basic.fundamental]).

Let `r` be `s % N`.

Returns: If `r` is 0, `x`; if `r` is positive, `(x << r) | (x >> (N - r))`; if `r` is negative, `rotr(x, -r)`.

```
template<class T>
[[nodiscard]] constexpr T rotr(T x, int s) noexcept;
```

Constraints: τ is an unsigned integer type (3.9.1 [basic.fundamental]).

Let r be $s \% N$.

Returns: If r is 0, x ; if r is positive, $(x \gg r) \mid (x \ll (N - r))$; if r is negative, $\text{rotr}(x, -r)$.

23.20.3 Counting [bitops.count]

```
template<class T>
constexpr int countl_zero(T x) noexcept;
```

Constraints: τ is an unsigned integer type (3.9.1 [basic.fundamental]).

Returns: The number of consecutive 0 bits in the value of x , starting from the most significant bit. [Note: Returns N if $x == 0$.]

```
template<class T>
constexpr int countl_one(T x) noexcept;
```

Constraints: τ is an unsigned integer type (3.9.1 [basic.fundamental]).

Returns: The number of consecutive 1 bits in the value of x , starting from the most significant bit. [Note: Returns N if $x == \text{std::numeric_limits}<T>::\text{max}()$.]

```
template<class T>
constexpr int countr_zero(T x) noexcept;
```

Constraints: τ is an unsigned integer type (3.9.1 [basic.fundamental]).

Returns: The number of consecutive 0 bits in the value of x , starting from the least significant bit. [Note: Returns N if $x == 0$.]

```
template<class T>
constexpr int countr_one(T x) noexcept;
```

Constraints: τ is an unsigned integer type (3.9.1 [basic.fundamental]).

Returns: The number of consecutive 1 bits in the value of x , starting from the least significant bit. [Note: Returns N if $x == \text{std::numeric_limits}<T>::\text{max}()$.]

```
template<class T>
constexpr int popcount(T x) noexcept;
```

Constraints: τ is an unsigned integer type (3.9.1 [basic.fundamental]).

Returns: The number of 1 bits in the value of x .

References

- ISO/IEC JTC1 SC22 WG21 [N3864](#): "A constexpr bitwise operations library for C++" by Matthew Fioravante
- ISO/IEC JTC1 SC22 WG21 [P0214R2](#): "Data-Parallel Vector Types & Operations" by Matthias Kretz
- ISO/IEC JTC1 SC22 WG21 [P0237R0](#): "On the standardization of fundamental bit manipulation utilities" by Vincent Reverdy and Robert J. Brunner
- ISO/IEC JTC1 SC22 WG21 [P0237R4](#): "Wording for fundamental bit manipulation utilities" by Vincent Reverdy and Robert J. Brunner
- ISO/IEC JTC1 SC22 WG21 [P0476R1](#): "Bit-casting object representations" by JF Bastien
- [ARM Compiler User Guide](#) [large PDF]
- [Intel x86 Instruction Set Reference](#) [large PDF]
- PowerPC Architecture Book, Version 2.02: [Book I: PowerPC User Instruction Set Architecture](#) [large PDF inside ZIP]